

PROJEKT ENGINEERING

Design

Herwig Mayr

Fakultät für Informatik, Kommunikation und Medien
Fachhochschule OÖ, Hagenberg

Design – zwischen Anforderungsanalyse und Implementierung



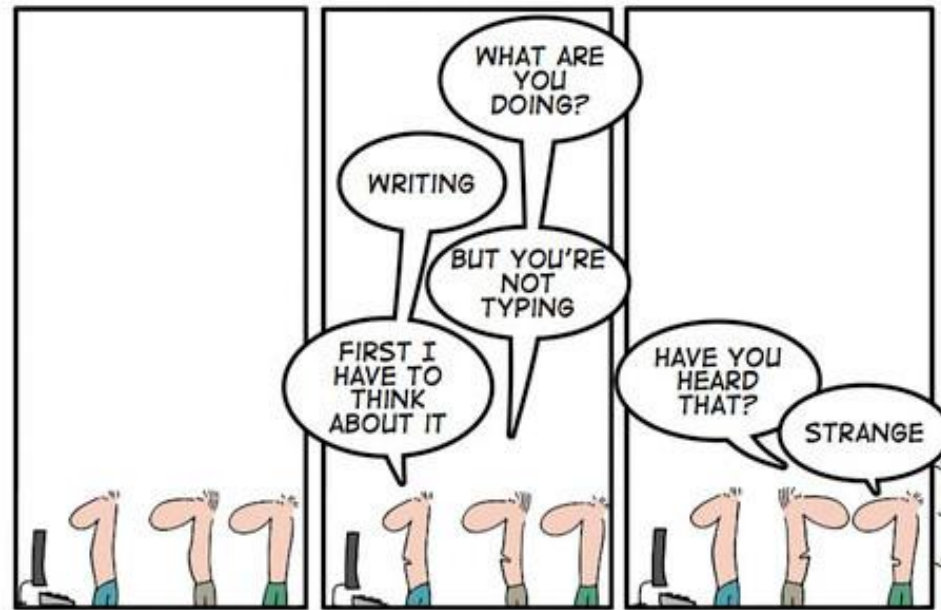
(Grafik: www.dilbert.com)

Design Ziel

Design – Ziel

Design legt fest, **WIE** die Anforderungen für die Implementierung aufbereitet werden.

-> Codieren soll möglichst klar strukturiert und in einfachen Schritten möglich sein!



(Grafik: geekandpoke.typepad.com)

Design Aufgaben

Design – Aufgaben

Erstellung des **Designmodells**, das die **physische Sicht** (Aufbau) und **logische Sicht** (Verhalten) des zu entwickelnden Systems beschreibt

Das Designmodell umfasst:

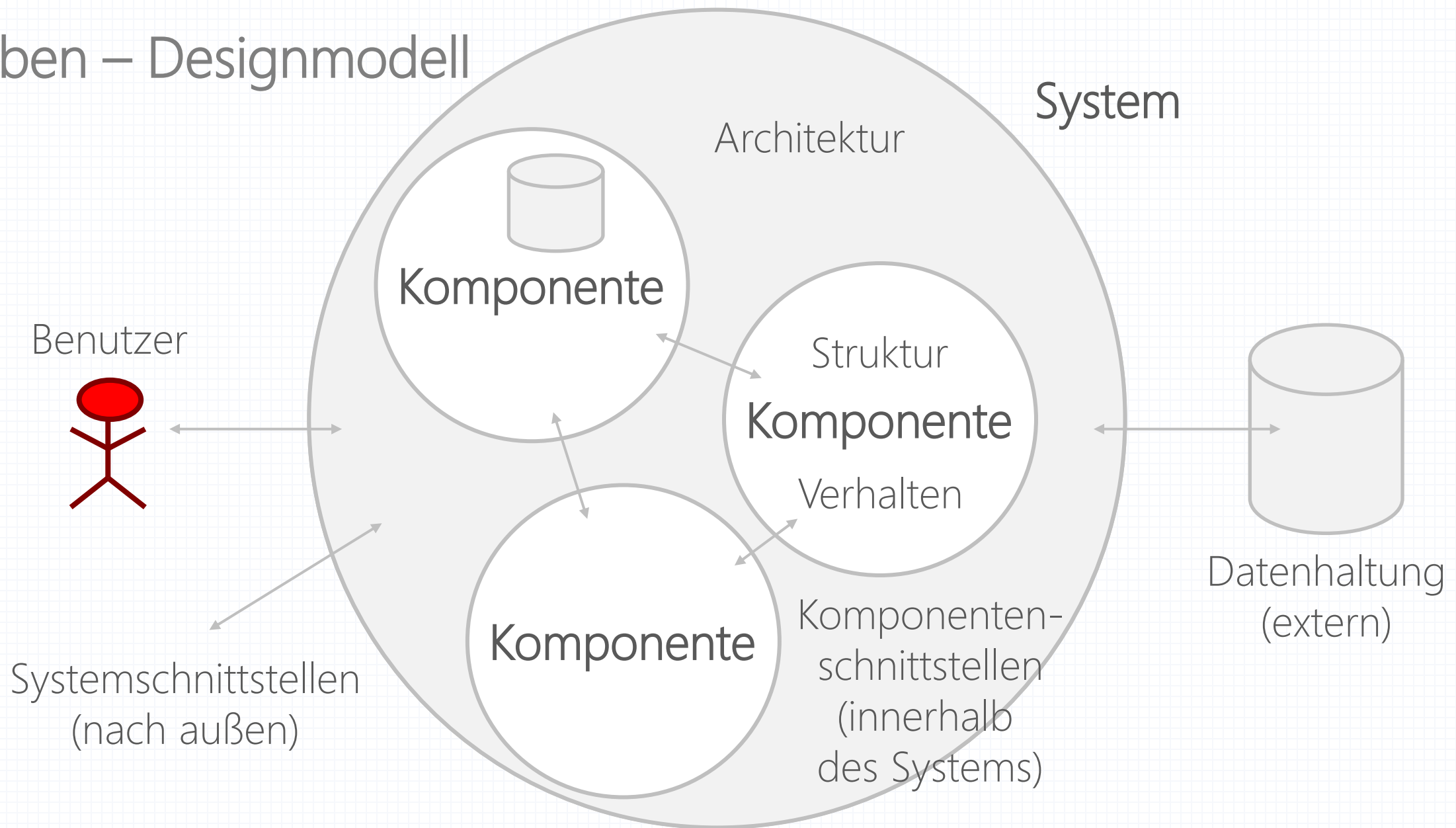
- Systemmodell
- Komponentenmodell

Dementsprechende Aufgaben:

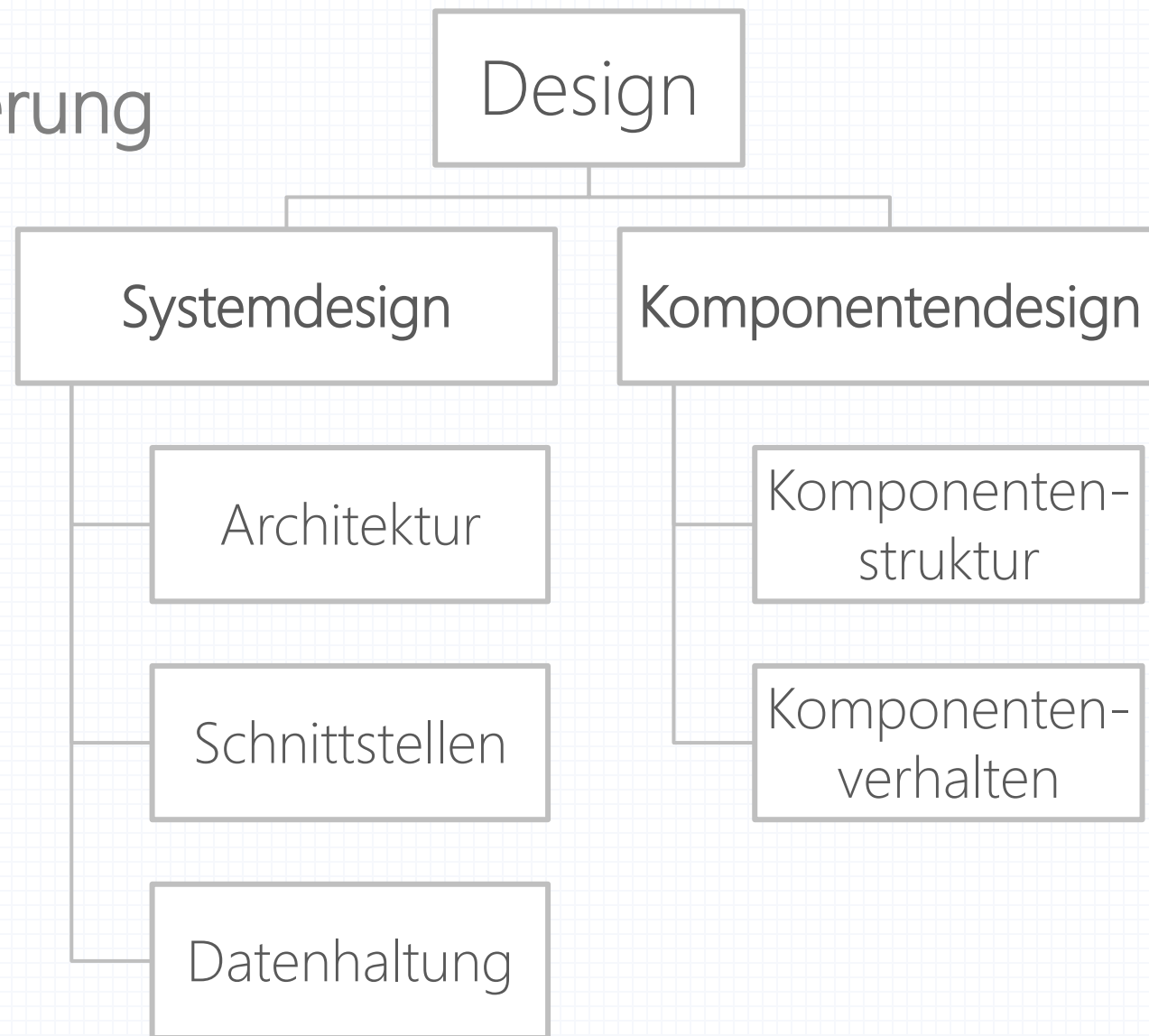
- Erstellen des **Systemdesigns**
- Erstellen des **Komponentendesigns**

Vorgehen nach **schrittweiser Verfeinerung** (Wirth)

Aufgaben – Designmodell



Aufgaben – Gliederung



-> Prioritäten festlegen, Kompromisse schließen!

Beispiel/Übung: Designaufgaben im LEGO-Projekt

- Architektur:
- Schnittstellen:
- Datenhaltung:
- Komponentenstruktur:
- Komponentenverhalten:

Systemdesign – Modellierung der Architektur

auch **Architekturdesign**; vielfach durch evolutionäre Prototypen

Erweiterung des Strukturmodells

- Entwickeln der Systemarchitektur aus dem Strukturmodell der Analyse (Analysebeschränkungen und Implementierungsvorgaben beachten)
- wiederzuverwendende Komponenten/Untersysteme einplanen

Erweiterung des Verhaltensmodells

- Zerlegung in Untersysteme inkl. deren zeitlichem Zusammenspiel (logische Systemgliederung in physische Untersysteme umsetzen -> verteilte Systeme)
- danach Abbildung auf Prozesse (Nebenläufigkeiten, Echtzeitverhalten beachten!)

Systemdesign – Modellierung der Schnittstellen

Systemschnittstellen werden nur durch Aufbau und Protokoll beschrieben (Details in der Implementierung).

Ausnahme: Echtzeitsysteme („Embedded Systems“)

Sonderstellung für **Benutzerschnittstellen**:

- Die vollständige Beschreibung aller Interaktionsabläufe erfolgt bereits im Design.
- Wegen dynamischer Abhängigkeiten ist Benutzerschnittstellen-Prototyping sehr sinnvoll.
- Die Ergonomie der Benutzerschnittstellen („Usability“) ist sehr wichtig (vgl. Qualitätsmerkmale für „Quality-in-Use“).

Systemdesign – Modellierung der Datenhaltung

Organisation des Ressourcenmanagements –
v. a. saubere Modellierung der Datenhaltung
und -ablage wichtig:

- **Datenstrukturen** im Hauptspeicher
- **Dateien** im Dateisystem / Netzwerk / Cloud
- **Datenbanken** inkl. Anbindung (zentral / verteilt)

(Grafik: Rob Cottingham)

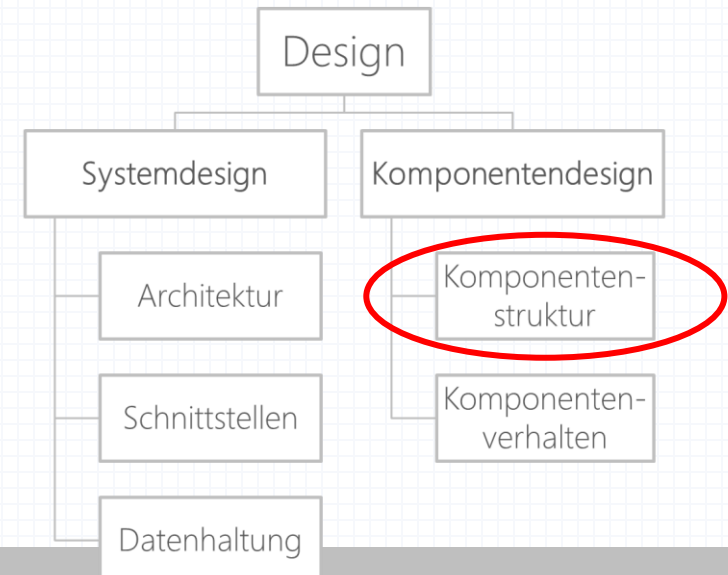


For the last time, this is Hell!
We don't have WiFi!

Komponentendesign – Modellierung der Struktur

Festlegung der **Komponentenstruktur**:

1. zuerst Abhängigkeiten und Schnittstellen zwischen den Komponenten festlegen und Kontrollfluss spezifizieren
2. wieder zu verwendende Komponenten auswählen
3. Komponentendefinition aus Analyse vervollständigen (wenn notwendig Strukturen ergänzen)
4. iterativ Strukturmodell adaptieren (abstrakte Komponenten einführen)



Komponentendesign – Modellierung des Verhaltens

Festlegung des **Komponentenverhaltens**:

- Beschreibung der Abläufe (Methoden)
- normalerweise im Design nur **deklarativ**, z. B.:
 - *uses*: Eingabeparameter
 - *changes*: Ausgabe-/Übergabeparameter
 - *informs*: erzeugte Nachrichten
 - *invariant*: Invarianten
 - *requires*: Vorbedingungen
 - *ensures*: Nachbedingungen
 - *produces*: Rückgabewerte
- nur bei schwierigen Algorithmen oder Performanzproblemen **funktionale** Beschreibung
- zusätzlich **Schnittstellenbeschreibung** (in UML Schnittstellenklassen – „Interfaces“ – und Lollipop-Notation)



Design Techniken

Design – Techniken

Techniken

- Erweitern der Analysemodelle
- Erstellen von Prototypen
- Modellgesteuertes Entwickeln
- Endbenutzer-Entwicklung

Techniken – Erweitern der Analysemodelle

Zweck: Analysemodell vom Problemraum in den Lösungsraum übertragen

Vorgehen:

- **traditionelles Vorgehen:** „vollständige“ Erstellung des Designs
- modernes, **agiles Vorgehen:** möglichst einfaches Design
– aber Achtung auf gutes Design-Grundgerüst (Big-Picture-Design)
- Generalisierung und Promotion von Wissen (erzeugt **Taxonomien**)
- **Topdown-Design** versus **Bottomup-Design** – in der Praxis gemischt

Beispiel/Übung: Designalternativen im LEGO-Projekt

Topdown-Design:

Bottomup-Design:



Techniken – Erstellen von Prototypen

vor allem **experimentelles Prototyping** und **evolutionäres Prototyping** für

- Architekturdesign und
- Benutzerschnittstellendesign

Prototyping ist Grundlage aller iterativ-inkrementellen („agilen“) Vorgehensmethoden!

(Grafik: marketoonist.com)



Beispiel/Übung: Einsatz von Prototyping

Experimentelles Prototyping:

Evolutionäres Prototyping:



Techniken – Modellgesteuertes Entwickeln (I)

„**Model-Driven Development**“ (MDD): reales System wird durch *reine* Transformationen aus einem Modell erzeugt

Basis: **Model-Driven Architecture** (MDA); diese ist Teil des UML2-Standards.
Bedenke: UML2 ist eine *vollständige* Programmiersprache!

Vorteile:

- plattformunabhängige Entwicklung
- Simulation auf Modellbasis möglich
- Plattform-Deployment automatisierbar



Techniken – Modellgesteuertes Entwickeln (II)

Vorgehensweise:

1. Erfasse Anforderungen, Modelle und Prozesse in einem plattformunabhängigen Modell (Platform-Independent Model – PIM).
2. Lege Rahmenbedingungen als Metadaten fest (Meta Object Facility – MOF).
3. Erzeuge daraus ein plattformspezifisches Modell (Platform-Specific Model – PSM).
4. Generiere ausführbaren Code vollautomatisch.

Techniken – Endbenutzer-Entwicklung

engl. End User Development
(auch: „Low Code / No Code Development“)

vor allem durch Maßschneidern und Befüllen von
Systemrahmen (Application Frameworks)

führt vielfach zu schlechten Ergebnissen (Unerfahrenheit
der Benutzer bzgl. gutem Designs, speziell bei Architektur,
aber auch bei Richtlinien zur Benutzerinteraktion)



Techniken – Wiederverwendung (I)

„Development by Investment“ – Wiederverwendbarkeit muss gegeben sein!

Vorgehen:

- Wiederverwenden von Bausteinen
- Abwandeln von Mustern
- Verwenden von Schablonen
- Konkretisieren von Systemrahmen



Techniken – Wiederverwendung (II)

Wiederverwenden von Bausteinen

= unverändertes Verwenden vorhandener Komponenten:

- Toolkits
- Modulbibliotheken
- Klassenbibliotheken

Abwandeln von Mustern

= Anwenden schematischer Lösungsvorschläge:

- Designmuster
- Analysemuster
- Prozessmuster

Techniken – Wiederverwendung (III)

Verwenden von Schablonen

= Erzeugen von Klassen/Objekten mittels

- abstrakter Klassen
- Schnittstellenklassen

Komponente = Zusammenstellung von eng gekoppelten Klassen (z.B. JavaBeans); aktive Komponenten besitzen eigene Intelligenz („**intelligente Agenten**“)

Konkretisieren von Systemrahmen

= Erweitern/Adaptieren einer Sammlung von Objekten, die die Grundfunktionalität einer Anwendung bieten („**generische Anwendung**“)

-> ermöglicht Wiederverwendung des Systemdesigns und Kontrollflusses

Beispiel/Übung: Arten der Wiederverwendung

- Bausteine:
- Muster:
- Schablonen:
- Systemrahmen:

Techniken – Empfehlungen für ein gutes Design

- Entwickle modulare Systeme.
- Eine Klasse, eine Abstraktion; eine Methode, eine Funktionalität.
- Befolge das Geheimnisprinzip.
- Begrenze die Tiefe von Vererbungsbäumen.
- Betreibe sinnvolle Wiederverwendung.
- Verwende Designmuster und Systemrahmen.
- Halte die Anzahl der Objektinteraktionen gering.



red**dot** design award



**GOOD DESIGN
AWARD 2013**



Design Werkzeuge

Design – Werkzeuge

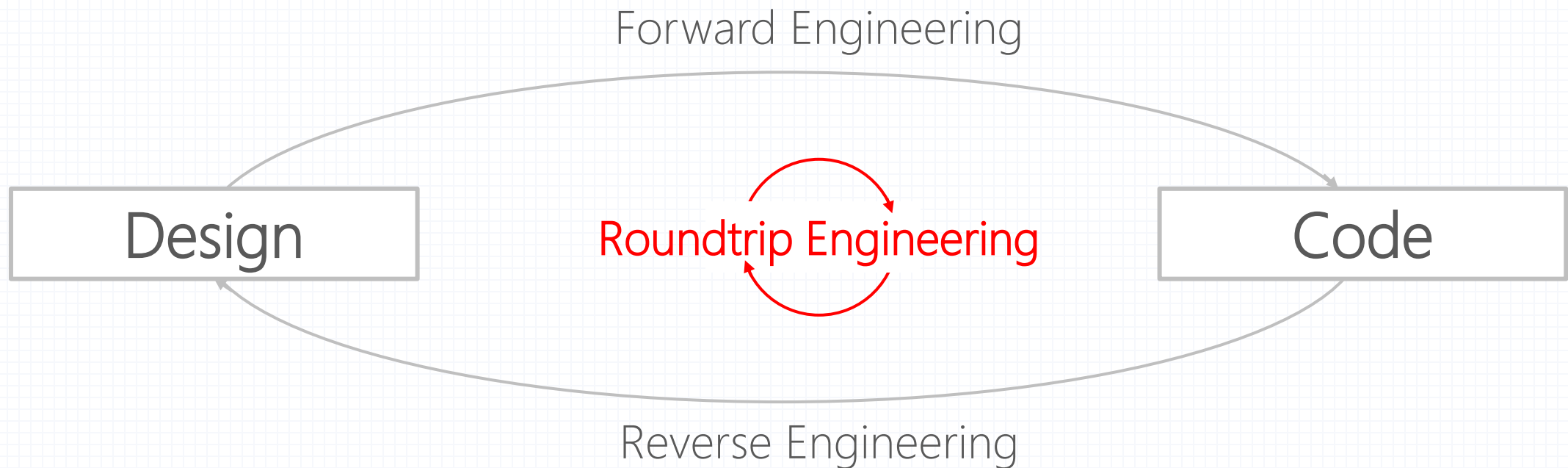
Werkzeuge

- Computergestützte Designwerkzeuge
- Unified Modeling Language (UML)
- Prototyping

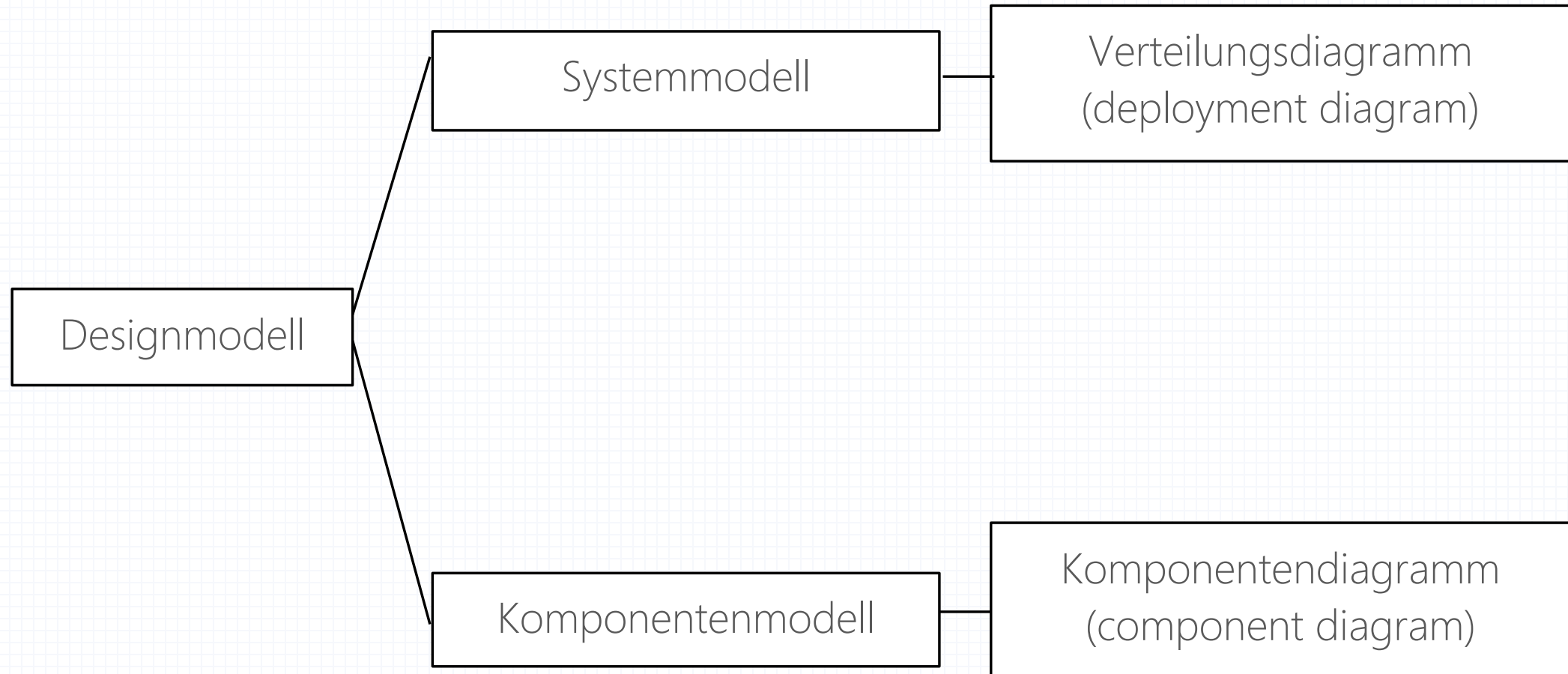
Werkzeuge – Computergestützte Designwerkzeuge

Die meisten Software-Entwicklungssysteme unterstützen auch das Design (sowie die Analyse).

vgl. **Forward Engineering** – **Reverse Engineering** – **Roundtrip Engineering**



Werkzeuge – Diagramme der UML für das Design (vereinfacht)



Werkzeuge – Vorgehen beim Design unter Verwendung der UML (I)

Entwickeln des Systemmodells:

1. Implementierungsbeschränkungen festlegen
2. Architektur mit Hardware/Systemsoftware abstimmen (Verteilungsdiagramm)
3. Datenhaltung organisieren; Persistenz! (Verteilungsdiagramm)
4. Nebenläufigkeit festlegen (Sequenzdiagramm)
5. Benutzerschnittstelle gestalten (Prototypen)

Werkzeuge – Vorgehen beim Design unter Verwendung der UML (II)

Entwickeln des Komponentenmodells:

1. Komponentenschnittstellen festlegen (Komponentendiagramm)
2. wiederverwendete Komponenten festlegen (Komponentendiagramm)
3. Strukturmodell verfeinern (Klassen-/Objektdiagramm)
4. Attribute/Operationen ergänzen (Klassen-/Objektdiagramm)
5. Gemeinsamkeiten ableiten (Klassen-/Objektdiagramm)
6. Abhängigkeiten realisieren (Klassen-, Kollaborationsdiagramm)
7. Kontrollfluss festlegen (Kollaborationsdiagramm)
8. Steuer-/Zustandsvariablen eintragen (Klassendiagramm)
9. Operationen spezifizieren (Zustands-/Aktivitätsdiagramm)

Wichtig: Festlegen von **Designkompromissen** und **Designprioritäten** nötig!

Werkzeuge – (Benutzerschnittstellen-)Prototyping

- **Beschreibungssprachen für Benutzerschnittstellen:**
textuelle Beschreibung des statischen Teils einer Benutzeroberfläche
- **Werkzeuge des Compilerbaus:**
vielfach ist Programmablauf durch Strom der Eingabedaten gesteuert;
Compilerbauwerkzeuge erstellen Parser etc.
- **Grafische Werkzeuge zur Erstellung von Benutzerschnittstellen:**
interaktive Eingabemöglichkeiten; oft kann Ablauf bereits simuliert werden



Design Ergebnisse

Design – Ergebnisse

Ergebnisse

- Designbericht
- Pflichtenheft
- Benutzerdokumentation

Erst seit 2014 rein elektronische Version zulässig!

(<https://www.it-recht-kanzlei.de/bedienungsanleitung-deutsche-sprache-produkte.html>)

Ergebnisse – Designbericht

Designbericht

- Erweiterung des Analyseberichts um die Designmodelle und um die Prototypen
- normalerweise intern

Ergebnisse – Pflichtenheft (I)

Pflichtenheft (bei IEEE: „System/Software Design Specification“ – SDS)

- ist „die Beschreibung der Realisierung aller Anforderungen des Lastenhefts“ (VDI).
- Es wird also definiert, **WIE** und **WOMIT** die Anforderungen zu realisieren sind.

Das Pflichtenheft wird **vom Auftraggeber abgenommen**; – bei agilem Vorgehen ggf. Alternativen überlegen (Abnahme von Zwischenprodukten, Protokollen etc.).

Ergebnisse – Pflichtenheft (II)

Pflichtenheft

Gliederungsvorschlag (gemäß VDI):

- wie Lastenheft, nur Adaptierung und Ergänzung um das Kapitel „Design“

Checkliste für das Pflichtenheft

zur Vorprüfung durch den Auftragnehmer (auch Lastenheft-Checkliste durchgehen!):

- Kompatibilität
- technische Qualität
- Realisierbarkeit

ICS 25.040.01, 35.240.50

VDI/VE-RICHTLINIEN

Januar 2008
January 2008

VEREIN
DEUTSCHER
INGENIEURE

VERBAND DER
ELEKTROTECHNIK
ELEKTRONIK
INFORMATIONSTECHNIK

Lastenheft/Pflichtenheft für den Einsatz von
Automatisierungssystemen

System requirement/specification for planning and
design of automation systems

VDI/VE 3694

Ausg. deutsch/englisch
Issue German/English

Die deutsche Version dieser Richtlinie ist verbindlich.

The German version of this guideline shall be taken as authoritative. No guarantee can be given with respect to the English translation.

Inhalt	Seite	Contents	Page
Vorbemerkung	2	Preliminary note	2
Einleitung	2	Introduction	2
1 Anwendungsbereich	2	1 Scope	2
2 Begriffe	3	2 Terms and definitions	3
3 Gliederung Lastenheft	3	3 Outline of system requirement	3
1 Einführung in das Projekt	4	1 Introduction into the project	4
2 Beschreibung der Ausgangssituation (Istzustand)	6	2 Description of initial situation (actual state)	6
3 Aufgabenstellung (Sollzustand)	8	3 Problem definition (specified state)	8
4 Anforderungen für die Kommunikationsschnittstellen	12	4 Communication interfaces requirements	12
5 Anforderungen an die Systemtechnik	13	5 Systems engineering requirement	13
6 Anforderungen für Systementwicklung, -inbetriebnahme und -einsatz	14	6 Requirements regarding system development, commissioning and use	14
7 Anforderungen an die Qualität	17	7 Quality requirements	17
8 Anforderungen an die Projektabwicklung	18	8 Project management requirements	18
4 Gliederung Pflichtenheft	19	4 Outline of specification	19
9 Systemtechnische Lösung	19	9 Systems engineering solution	19
10 Systemtechnik (Ausprägung)	20	10 Systems engineering specifics (realisation)	20
5 Anhang zum Lastenheft/ Pflichtenheft	21	5 Annex to the system requirement/ specification	21
A1 Begriffe	21	A1 Terms and definitions	21
A2 Abkürzungen	21	A2 Abbreviations	21
A3 Nomenklatur	21	A3 Nomenclature	21
A4 Gesetze, Normen, Richtlinien	21	A4 Acts, standards, guidelines	21
Schrifttum	22	Bibliography	22

VDI/VE-Gesellschaft Mess- und Automatisierungstechnik (GMA)

Fachausschuss Durchgängiges Engineering von Leitsystemen

VDI/VE-Handbuch Mess- und Automatisierungstechnik, Band 1: Prozesstechnisches Messen
 VDI/VE-Handbuch Mess- und Automatisierungstechnik, Band 2: Fertigungstechnisches Messen
 VDI/VE-Handbuch Mess- und Automatisierungstechnik, Band 3: Automatisierungstechnik
 VDI-Handbuch Energietechnik

VDI-Handbuch Materialfluss und Fördertechnik, Band 7: Materialfluss I (Gestaltung)

VDI-Handbuch Technischer Vertrieb und Produktmanagement

VDI-Handbuch Zuverlässigkeit

Zuerst erschienen durch VDI-Verlag, Düsseldorf, 1977/2008. Alle Rechte vorbehalten. / All rights reserved © Verein Deutscher Ingenieure e.V., Düsseldorf 2008.
 Former edition: 04/91/06/05 Entwurf, deutsch
 Former edition: 04/91/06/05 Draft, in German only

Vervielfältigung – auch für innerbetriebliche Zwecke – nicht gestattet / Reproduction – even for internal use – not permitted

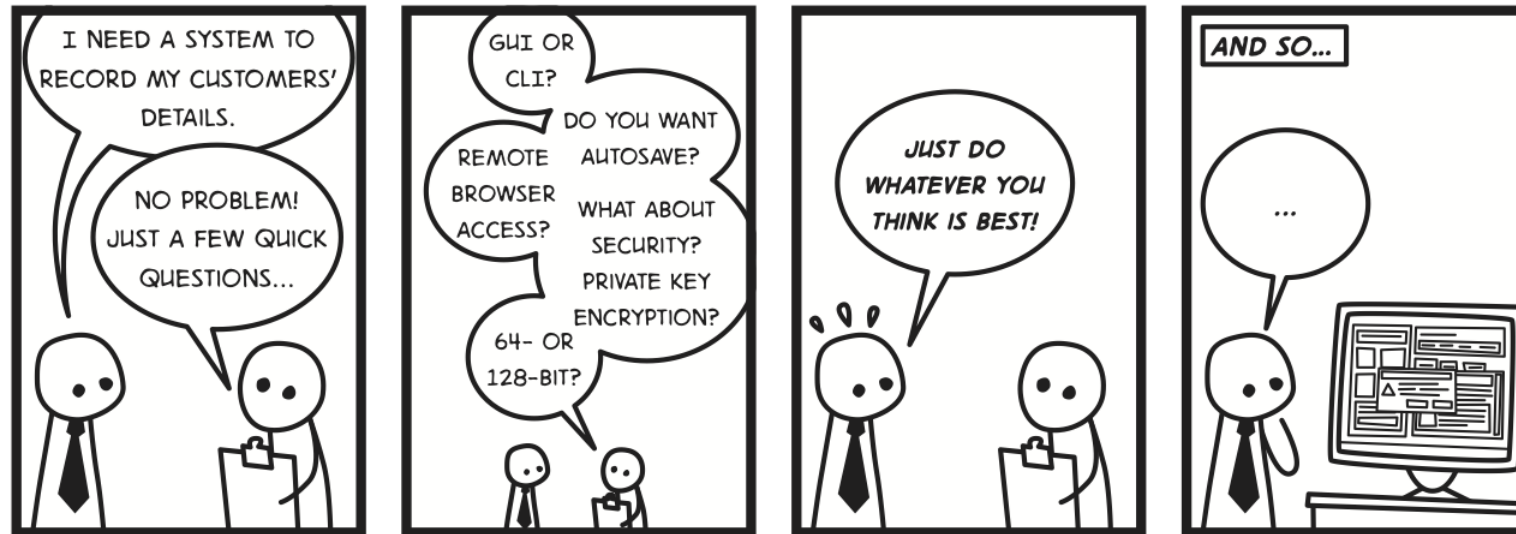
Beispiel/Übung: Erstellen eines Pflichtenhefts für das LEGO Projekt

-> Übung!

Ergebnisse – Benutzerdokumentation (I)

Benutzerdokumentation

- erste Version bereits während des Designs erstellt, um die Designergebnisse bei der Implementierung leicht zugänglich zu halten
- bereits Reviews mit dem Auftraggeber möglich



(Grafik: www.cosc.canterbury.ac.nz)

Ergebnisse – Benutzerdokumentation (II)

Komponenten

- Allgemeine Produktbeschreibung
- Installationsanleitung
- Anweisungen zur Inbetriebnahme
- Kurzeinführung (Tutorial)
- Benutzerhandbuch
- Referenzhandbuch
- Referenzkarte (Quick Reference Guide)
- Online-Hilfe
- Supplement, Readme-Datei



Ergebnisse – Benutzerdokumentation (III)

Checkliste für die Benutzerdokumentation

- Vollständigkeit
- Verständlichkeit
- Strukturiertheit
- Modularität
- Benutzbarkeit
- Aktualität



Unstimmigkeiten zwischen System und Dokumentation werden vom Auftraggeber am raschesten erkannt und sind kaum zu dementieren.
-> Sorgfältig prüfen!

Eine Benutzerdokumentation kann nicht zu einfach sein!



PROJEKT ENGINEERING

Design

Herwig Mayr

Fakultät für Informatik, Kommunikation und Medien
Fachhochschule OÖ, Hagenberg